

Endbericht zur Projektaufgabe „Mobile Roboter“ im Rahmen der Vorlesung „Innovative Konzepte zur Programmierung von Industrierobotern“

Martin Filpe, Luca Beck, Christian Diehm – 25.07.2025

Einführung und Problemstellung

Mobile Roboter spielen in vielen Lebens- und Arbeitsbereichen eine zunehmend wichtige Rolle. Zur Unterstützung von Kunden in Supermärkten, beim Materialtransport in der Industrie oder in Pflegeeinrichtungen werden mobile Roboter eingesetzt, um Menschen zu unterstützen oder zu ersetzen. Die Planung mobiler Roboter ist hierbei eine der größten Herausforderungen. Im Rahmen unserer Projektaufgabe verwendeten wir vier aus der Vorlesung bekannten probabilistischen Planungsverfahren, um holonome Roboter mit zwei bzw. drei Freiheitsgraden in einer zweidimensionalen Umgebung mit statischen Hindernissen zu planen. Dabei durften die Planungsfahren nicht verändert werden. Holonom bedeutet in diesem Zusammenhang, dass der Roboter sich jederzeit in alle Richtungen des Konfigurationsraums bewegen kann. In unserem Fall können sich die Roboter in der x/y-Ebene frei bewegen und ggf. frei um die z-Achse drehen. Zur Umsetzung wurden verschiedene Roboter sowie Benchmark-Umgebungen modelliert, ein Kollisionsprüfer entwickelt und zusammen mit ihren Auswertungen in einem Jupyter Notebook integriert.

Vorgehensweise

Für die Kollisionsprüfung prüfen wir, ob das transformierte Roboter-Polygon mit einem Hindernis-Polygon kollidiert. Da die Robotergeometrie nur im lokalen Koordinatensystem vorliegt, wird vor der Kollisionsprüfung jeweils eine Transformation an die aktuelle Roboterposition und -orientierung durchgeführt. Zur Verbesserung der Laufzeit bei mehreren Robotern wurde ein vorgelagerter AABB-Test (Axis-Aligned Bounding Boxes) eingesetzt. Zudem wurden auch Kollisionen zwischen den Robotern berücksichtigt, indem sich diese gegenseitig als dynamische Hindernisse behandeln. Zur Prüfung, ob eine Bewegung zwischen zwei Positionen im Konfigurationsraum kollisionsfrei ist, wird der Pfad linear interpoliert und für 41 Zwischenpunkte jeweils überprüft, ob eine Kollision mit einem Hindernis vorliegt.

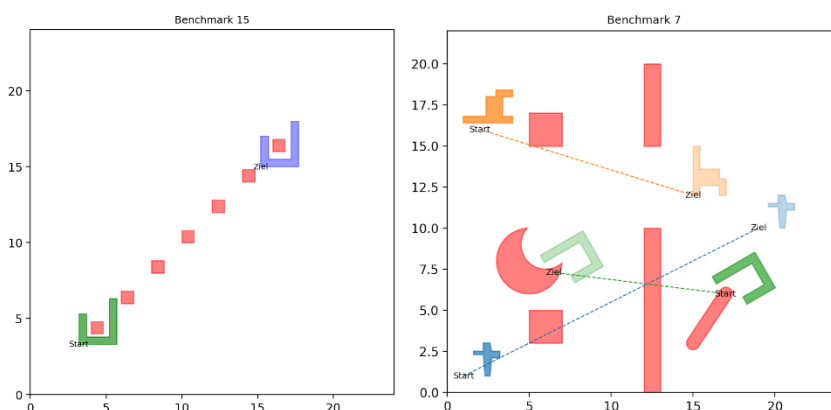


Abbildung 1 - Benchmark-Umgebungen mit einem bzw. mehreren mobilen Robotern

Für die Auswertung wurden sechs Benchmark-Umgebungen erstellt, in denen die Hindernisse gezielt platziert wurden. Um eine gute Vergleichbarkeit sicherzustellen, haben wir Start- und Zielpositionen sowie die Anzahl der Freiheitsgrade der Roboter manuell festgelegt. Unser Code bietet zusätzlich die

Möglichkeit, zufällige, kollisionsfreie Start- und Zielpunkte zu erzeugen sowie die Anzahl der Roboter und deren Freiheitsgrade beliebig festzulegen. Abbildung 1 zeigt ein Beispiel für einen Einzelroboter-Benchmark mit engem Abstand zu einem Hindernis an Start und Ziel sowie ein Szenario mit drei Robotern, in dem die Herausforderung darin besteht, dass sie sich innerhalb eines dicht besetzten Hindernisfelds kollisionsfrei aneinander vorbeibewegen müssen.

Ergebnisse

Insgesamt wurden 30 Benchmarks für einzelne Roboter untersucht, aufgeteilt in fünf verschiedene Szenen mit jeweils sechs Robotern, davon drei mit zwei Freiheitsgraden (2DoF) und drei mit drei Freiheitsgraden (3DoF). Zusätzlich wurden zehn Multi-Robot-Benchmarks erstellt: fünf mit zwei Robotern und fünf mit dreien.

Im Folgenden wird exemplarisch ein Benchmark gezeigt, bei dem ein 2DoF-Roboter an zwei Hindernissen vorbeinavigieren muss. Zusätzlich sind die jeweiligen Konfigurationsräume der jeweiligen Planungsverfahren dargestellt, um deren Verhalten nachvollziehbar zu machen.

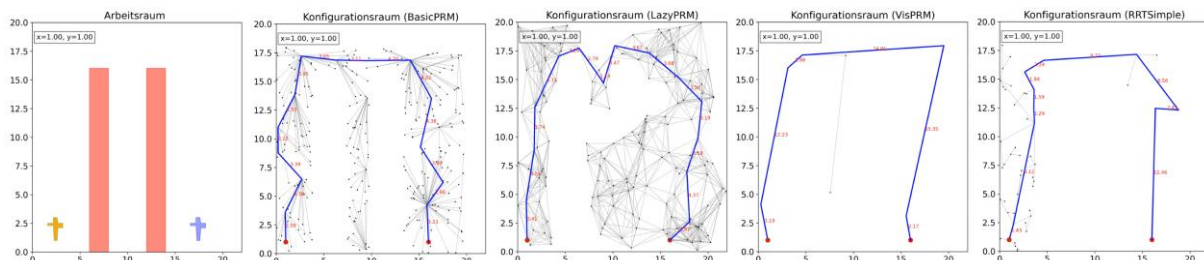


Abbildung 2 - Planung eines 2DoF Roboters mit den Planungsverfahren BasicPRM, LazyPRM, VisibilityPRM und RRT

Anhand der Konfigurationsräume lässt sich deutlich erkennen, dass BasicPRM ausschließlich kollisionsfreie Konfigurationen in die Roadmap aufnimmt. Im Gegensatz dazu verschiebt LazyPRM die Kollisionstests auf einen späteren Zeitpunkt, sodass in der Roadmap zunächst auch kollisionsbehaftete Konfigurationen enthalten sind. VisibilityPRM erzeugt eine kompakte Roadmap, da nur Verbindungen aufgenommen werden, die zur Erweiterung der Sichtbarkeit beitragen. Bei RRT zeigt sich ebenfalls das charakteristische Verhalten, bei dem sich die Struktur vom Startpunkt in den Raum ausbreitet.

	1 Roboter mit 2 Freiheitsgraden				1 Roboter mit 3 Freiheitsgraden			
	Zeit [s]	Roadmap Größe	# Kanten im Pfad	Länge Pfad	Zeit [s]	Roadmap Größe	# Kanten im Pfad	Länge Pfad
BasicPRM	1,21	218,9	14,5	44,8	1,98	294,9	19,8	352,0
LazyPRM	0,62	533,3	21,9	43,5	10,88	787,5	27,5	272,7
VisibilityPRM	0,55	14,7	7,6	51,2	0,99	24,1	10,3	838,0
RRT	0,21	48,3	10,5	44,7	0,99	170,5	10,1	356,1

Tabelle 1 – Gemittelte Ergebnisse der 30 Einzelroboter-Benchmarks.

Die Planungsverfahren zeigen in verschiedenen Metriken jeweils unterschiedliche Stärken. VisibilityPRM überzeugt mit kleinen Roadmaps und wenigen Kanten im Pfad, erzeugt dabei aber oft deutlich längere Pfade. LazyPRM hingegen liefert tendenziell kürzere Pfade, benötigt dafür jedoch deutlich mehr Zeit. Die Ergebnisse hängen ebenfalls stark von den gewählten Parametern der jeweiligen Planungsverfahren ab. Für die nun folgenden Multi-Robot-Benchmarks haben wir den Verbindungsradius bei BasicPRM sowie den kNearest-Wert bei LazyPRM anhand analytischer Formeln angepasst, abhängig von der Dimensionalität des Konfigurationsraums, den Koordinatenlimits und der Anzahl der Knoten. Die Berechnung basiert auf den Empfehlungen von Karaman und Frazzoli [1], die in ihrer Arbeit die Asymptotisch-Optimalität untersuchen und entsprechende Parameter ableiten.

	2 Roboter mit insg. 5 Freiheitsgraden				3 Roboter mit insg. 8 Freiheitsgraden			
	Zeit [s]	Roadmap Größe	# Kanten im Pfad	Länge Pfad	Zeit [s]	Roadmap Größe	# Kanten im Pfad	Länge Pfad
BasicPRM	2,07	302,0	16,6	1912,5	13,71	497,0	20,4	3704,6
LazyPRM	44,58	2035,6	16,4	222,6	-	-	-	-
VisibilityPRM	9,64	93,8	12,0	950,1	-	-	-	-
RRT	2,72	280,2	10,8	276,1	54,09	987,2	11,2	644,0

Tabelle 2 – Gemittelte Ergebnisse der zehn Multi-Roboter-Benchmarks

Die Ergebnisse der Multi-Robot-Benchmarks zeigen deutlich, wie stark der Rechenaufwand mit wachsendem Konfigurationsraum ansteigt. Bei zwei Robotern mit insgesamt fünf Freiheitsgraden liefert BasicPRM die schnellste Laufzeit, allerdings mit sehr langen Pfaden. LazyPRM findet zwar sehr kurze Pfade, benötigt dafür aber mit Abstand die meiste Rechenzeit. RRT zeigt ein insgesamt robustes Verhalten mit akzeptablen Pfadlängen sowie geringen Laufzeiten bei zwei Robotern. Bei drei Robotern ist ein deutlicher Anstieg der Rechenzeit zu erkennen. Während BasicPRM noch innerhalb akzeptabler Zeit einen Pfad findet, blieben LazyPRM und VisibilityPRM auch nach umfangreicher Parametrierung und Finetuning ohne Ergebnis.

Ein möglicher Grund für das Fehlschlagen von VisibilityPRM liegt an der Betrachtung von Kollisionen zwischen den Robotern untereinander. Durch die so entstehende dynamische Szenenumgebung entsteht eine unbeherrschbar hohe Fragmentierung des Konfigurationsraums. LazyPRM hingegen verzögert die Kollisionstests so lange wie möglich, was im 8D-Raum zu vielen unnötigen Verbindungen und damit ineffizienten Suchprozessen führt. Der Aufwand für die Kollisionsprüfung wächst exponentiell mit der Anzahl der beteiligten Roboter. Beide Verfahren stoßen damit in solchen hochdimensionalen Multi-Robot-Szenarien an praktische Grenzen.

Für die Planung mehrerer Roboter müssen zudem einige Annahmen für die Ausführung der Lösung getroffen werden. Beispielsweise wird bei unserer Planung angenommen, dass alle Roboter ihre Bewegung gleichzeitig beginnen, ihre Wegpunkte erreichen und gleichzeitig enden. Zudem setzen die geplanten Bahnen eine perfekte Ausführung voraus, was insbesondere bei engen Passagen schwierig wird, da eine solch perfekte Ausführung in der Praxis oft nicht zuverlässig umsetzbar ist. Ebenfalls zeigen unsere Visualisierungen, dass sich die Geschwindigkeit und Position der Roboter entlang ihrer Bahnen teils abrupt ändert. Für die Ausführung ohne vorgelagertes Glätten muss also angenommen werden, dass die Roboter solche ruckartigen Änderungen in der Bewegungsbahn ohne Verzögerung umsetzen können.

Mögliche Erweiterungen

Eine mögliche Erweiterung wäre die Planung eines mobilen Roboters mit zusätzlichem Manipulator. In diesem Fall müsste neben der Positionsplanung des Roboters auch die Bewegung des Arms in einem erweiterten Konfigurationsraum berücksichtigt werden. Hierbei ergibt sich also, bei einem Manipulator mit n Freiheitsgraden, ein Konfigurationsraum von $(2+n)$ bzw. $(3+n)$ Freiheitsgraden. Durch mögliche Gelenkgrenzen und notwendige kinematischen Berechnungen erhöht sich zudem der Berechnungsaufwand deutlich. Eine sinnvolle Vorgehensweise wäre, solange der Manipulator nicht gleichzeitig mit der Plattform-Bewegung benötigt wird, zunächst eine kollisionsfreie Basisbewegung der Plattform zu planen. Hierbei bewegt sich der Manipulator nicht, wodurch sich der Konfigurationsraum auf die Plattform beschränkt. In der Nähe des Ziels kann nun der Manipulator eingesetzt werden, um die geforderte Endkonfiguration zu erreichen. Diese zweistufige Vorgehensweise ermöglicht

eine schnellere Berechnung, bei der die Plattform und der Manipulator getrennt voneinander betrachtet werden und der Konfigurationsraum nicht unnötig groß wird. Unsere Ergebnisse haben bereits gezeigt, dass ein wachsender Konfigurationsraum, wie etwa bei mehreren mobilen Robotern, zu einem teilweise nicht mehr beherrschbaren Berechnungsaufwand führt.

Zur Planung mehrerer mobiler Roboter können ebenfalls andere Ansätze herangezogen werden. Um den Konfigurationsraum möglichst gering und damit effiziente Planungen zu ermöglichen, eignet sich beispielsweise ein Greedy-Ansatz, bei dem die Roboter nacheinander geplant werden und jeweils versuchen, auf direktem Weg ihr Ziel zu erreichen. Dabei würden bereits geplante Pfade der anderen Roboter als dynamische Hindernisse berücksichtigt. Aufgrund der fehlenden Koordination zwischen den Robotern besteht hierbei jedoch die Gefahr von Zuständen, in denen ein Roboter einem anderen dauerhaft den Zugang zum Ziel blockiert. Weitere Möglichkeiten sind beispielsweise die Verwendung von Potentialfeldern, bei denen die Roboter durch attraktive Kräfte zum Ziel und durch abstoßende Kräfte von Hindernissen und anderen Robotern geleitet werden. Ebenso eignet sich ein Scheduling-Verfahren zur Planung, was den Vorteil hat, dass die Roboter ihre Punkte auf dem Weg nicht gleichzeitig erreichen müssen und eine gleichbleibende Geschwindigkeit erhalten könnten.

Aus unseren Ergebnissen geht hervor, dass viele der geplanten Bewegungsbahnen unnatürlich verlaufen und teilweise stark gekrümmte oder zackige Formen annehmen. Solche Bahnen sind zwar kollisionsfrei, aber oft ineffizient oder unrealistisch für reale Roboter. Zur Glättung können nachgelagerte Optimierungsschritte eingesetzt werden, etwa über Kurvenapproximation durch inverses Runden. Eine weitere Methode besteht darin, einzelne Wegpunkte (z.B. mit besonders spitzen Winkeln) aus der geplanten Bahn zu entfernen und das neue Teilstück (Shortcut) erneut auf Kollisionsfreiheit zu überprüfen.

Quellenverzeichnis

- [1] Karaman, S., & Frazzoli, E. (2011). Sampling-based Algorithms for Optimal Motion Planning. <http://arxiv.org/abs/1105.1186>